

Tarea clase 7 github tarea4

miércoles, 23 de septiembre de 2026 7:45

1. Ejercicios : Control de calificaciones

Desarrolle un programa que permita ingresar las calificaciones de N estudiantes.

El número de estudiantes debe ser mayor que cero y cada calificación deberá estar entre 0 y 10.

El programa deberá determinar:

- número de estudiantes;
- suma de calificaciones;
- promedio general;
- cantidad de aprobados;
- cantidad de reprobados;
- nota más alta;
- nota más baja.

1. Análisis

El programa solicita la cantidad de estudiantes N y sus respectivas calificaciones, validando mediante bucles while que N sea mayor que cero y que cada nota se encuentre estrictamente entre 0 y 10. Mediante un bucle for, recorre las N iteraciones acumulando la suma total de las notas, contando cuántos alumnos aprueban (nota ≥ 7) y cuántos reprueban (nota < 7), y actualizando dinámicamente los valores para la nota más alta y la nota más baja. Finalmente, calcula el promedio general dividiendo la suma total para N y despliega en pantalla el reporte estadístico completo del curso.

INICIO

1. Inicio

- Objetivo: Registrar las calificaciones de N estudiantes, validar que los datos estén en los rangos permitidos y calcular estadísticas generales del curso.
- Inicialización de Variables:
- aprobados = 0 (Contador)
- reprobados = 0 (Contador)
- suma = 0.0 (Acumulador)
- notaAlta = -1.0 (Para encontrar el valor máximo)
- notaBaja = 11.0 (Para encontrar el valor mínimo)

2. Entrada

- Cantidad de estudiantes (N):
- Se lee el valor de N.
- Validación (while): Si $N \leq 0$, se solicita nuevamente la entrada hasta que el usuario ingrese un entero positivo ($N > 0$).
- Calificaciones por estudiante:
- Dentro de un ciclo que se repite N veces, se lee la nota de cada estudiante.
- Validación (while): Si la nota es menor a 0 o mayor a 10, se solicita de nuevo la entrada hasta que esté en el rango [0, 10].

3. Proceso

Para cada nota válida ingresada en la iteración i (desde 1 hasta N usando un bucle for):

Suma:

-
- Conteo de aprobados y reprobados:
- Si $\text{nota} \geq 7.0$, se incrementa aprobados = aprobados + 1.
- Si no ($\text{nota} < 7.0$), se incrementa reprobados = reprobados + 1.
- Determinación de extremos (Máximo y Mínimo):
- Si $\text{nota} > \text{notaAlta}$, entonces $\text{notaAlta} = \text{nota}$.
- Si $\text{nota} < \text{notaBaja}$, entonces $\text{notaBaja} = \text{nota}$.
- Cálculo del Promedio General:
- Al finalizar el ciclo de las N notas, se calcula:

4. Salida

- Muestra en pantalla los siguientes resultados consolidados:
- Número de estudiantes: N
- Suma de calificaciones: Valor acumulado en suma
- Promedio general: Valor calculado en promedio
- Cantidad de aprobados: Contador aprobados
- Cantidad de reprobados: Contador reprobados
- Nota más alta: Valor guardado en notaAlta
- Nota más baja: Valor guardado en notaBaja

FIN

2.PSEUCODIGO

Algoritmo ControlDeCalificaciones

// 1. Declaración de variables

Definir n, i, aprobados, reprobados Como Entero

Definir nota, suma, promedio, notaAlta, notaBaja Como Real

// 2. Inicialización de contadores y acumuladores

aprobados <- 0

reprobados <- 0

suma <- 0

notaAlta <- -1 // Inicializado por debajo del mínimo posible (0)

notaBaja <- 11 // Inicializado por encima del máximo posible (10)

// 3. Validación de la cantidad de estudiantes (uso obligatorio de 'while')

Escribir "Ingrese el número de estudiantes (N > 0):"

Leer n

Mientras n <= 0 Hacer

Escribir "Error: El número de estudiantes debe ser mayor que cero."

Escribir "Ingrese nuevamente el número de estudiantes:"

Leer n

FinMientras

// 4. Bucle principal para el procesamiento de notas (uso obligatorio de 'for')

Para i <- 1 Hasta n Con Paso 1 Hacer

Escribir "Ingrese la calificación del estudiante ", i, " (entre 0 y 10):"

Leer nota

// Validación de rango de la nota (uso obligatorio de 'while')

Mientras nota < 0 O nota > 10 Hacer

Escribir "Error: La calificación debe estar entre 0 y 10."

Escribir "Ingrese nuevamente la calificación del estudiante ", i, " :"

Leer nota

FinMientras

// Acumular la suma de notas

suma <- suma + nota

// Conteo de aprobados y reprobados (criterio estándar: >= 7 aprueba)

Si nota >= 7 Entonces

aprobados <- aprobados + 1

Sino

reprobados <- reprobados + 1

FinSi

// Determinación de la nota más alta

Si nota > notaAlta Entonces

notaAlta <- nota

FinSi

// Determinación de la nota más baja

Si nota < notaBaja Entonces

notaBaja <- nota

FinSi

FinPara

```
// 5. Cálculo del promedio general
```

```
promedio <- suma / n
```

```
// 6. Presentación de resultados
```

```
Escribir "=====
```

```
Escribir " RESULTADOS DEL CONTROL DE CALIFICACIONES"
```

```
Escribir "=====
```

```
Escribir "Número de estudiantes: ", n
```

```
Escribir "Suma de calificaciones: ", suma
```

```
Escribir "Promedio general: ", promedio
```

```
Escribir "Cantidad de aprobados: ", aprobados
```

```
Escribir "Cantidad de reprobados: ", reprobados
```

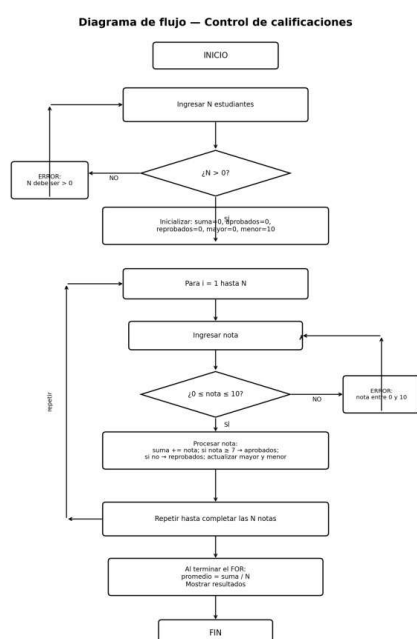
```
Escribir "Nota más alta: ", notaAlta
```

```
Escribir "Nota más baja: ", notaBaja
```

```
Escribir "=====
```

```
FinAlgoritmo
```

3.DIAGRAMA DE FLUJO



3.Código en Java

```
import java.util.Scanner;
```

```
public class ControlCalificaciones {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int n;
```

```
        int aprobados = 0;
```

```
        int reprobados = 0;
```

```
        double suma = 0;
```

```
        double notaAlta = Double.NEGATIVE_INFINITY;
```

```
        double notaBaja = Double.POSITIVE_INFINITY;
```

```
        // Validar número de estudiantes con 'while'
```

```
        System.out.print("Ingrese el número de estudiantes (N > 0): ");
```

```
        n = scanner.nextInt();
```

```
        while (n <= 0) {
```

```

        System.out.print("Error: El número de estudiantes debe ser mayor que cero. Intente de nuevo:
");
        n = scanner.nextInt();
    }

    // Procesar calificaciones con 'for'
    for (int i = 1; i <= n; i++) {
        System.out.print("Ingrese la calificación del estudiante " + i + " (0 a 10): ");
        double nota = scanner.nextDouble();

        // Validar rango de la calificación con 'while'
        while (nota < 0 || nota > 10) {
            System.out.print("Error: La calificación debe estar entre 0 y 10. Intente de nuevo: ");
            nota = scanner.nextDouble();
        }

        // Acumular y contar
        suma += nota;

        if (nota >= 7.0) {
            aprobados++;
        } else {
            reprobados++;
        }

        // Actualizar nota más alta y más baja
        if (nota > notaAlta) {
            notaAlta = nota;
        }
        if (nota < notaBaja) {
            notaBaja = nota;
        }
    }

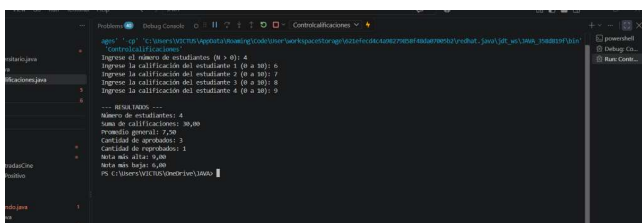
    // Calcular promedio general
    double promedio = suma / n;

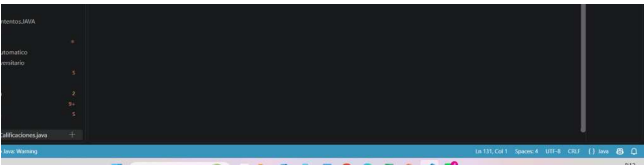
    // Mostrar resultados
    System.out.println("\n--- RESULTADOS ---");
    System.out.println("Número de estudiantes: " + n);
    System.out.printf("Suma de calificaciones: %.2f\n", suma);
    System.out.printf("Promedio general: %.2f\n", promedio);
    System.out.println("Cantidad de aprobados: " + aprobados);
    System.out.println("Cantidad de reprobados: " + reprobados);
    System.out.printf("Nota más alta: %.2f\n", notaAlta);
    System.out.printf("Nota más baja: %.2f\n", notaBaja);

    scanner.close();
}
}

```

CODIGO EN JAVA CORRIENDO





5.PRUEBAS DE ESCRITORIOS:

i	Nota	Suma	Aprobados	Reprobados	Nota alta	Nota baja
Inicial	—	0	0	0	-∞	+∞
1	8	8	1	0	8	8
2	6	14	1	1	8	6
3	10	24	2	1	10	6
4	7	31	3	1	10	6
5	5	36	3	2	10	5

Ejercicio2_tarea4

miércoles, 23 de septiembre de 2026 8:26

Ejercicio 2. Tabla de multiplicar configurable

Solicite dos números:

* tabla inicial;

* tabla final.

Por ejemplo:

* Tabla inicial: 3

* Tabla final: 5

El programa deberá generar:

TABLA DEL 3

3 x 1 = 3

...

3 x 10 = 30

TABLA DEL 4

...

TABLA DEL 5

...

Debe validar que la tabla inicial no sea mayor que la tabla final.

Estructura obligatoria: ciclos for anidados.

Desafío: permitir que el usuario determine también hasta qué multiplicador desea generar las tablas.

1. Análisis

El programa solicita al usuario dos valores enteros para definir el rango de las tablas de multiplicar (tablaInicial y tablaFinal), además de una entrada adicional opcional según el desafío para definir el multiplicador límite (por defecto hasta 10). Mediante una validación, se garantiza que tablaInicial no sea estrictamente mayor que tablaFinal. Posteriormente, se implementa una estructura de dos ciclos for anidados: el bucle externo itera desde la tabla inicial hasta la tabla final para imprimir los encabezados correspondientes, mientras que el bucle interno recorre los multiplicadores desde 1 hasta el límite deseado (ej. 10), calculando y desplegando el producto exacto en cada iteración.

Aquí tienes la estructura del Ejercicio 2: Tabla de multiplicar configurable organizada formalmente en Inicio, Entrada, Proceso, Salida y Fin:

1. Inicio

Objetivo: Generar y mostrar las tablas de multiplicar dentro de un rango especificado por el usuario (tablaInicial a tablaFinal), controlando opcionalmente el límite del multiplicador.

Declaración e Inicialización de Variables:

- tablaInicial (Entero)
- tablaFinal (Entero)
- multiplicadorLimite (Entero)
- i, j (Variables de control para ciclos)

2. Entrada

Rango de tablas:

- Solicitar e ingresar tablaInicial y tablaFinal.
- Validación (while): Verificar que tablaInicial <= tablaFinal. Si tablaInicial > tablaFinal, mostrar mensaje de error y solicitar ambos datos nuevamente.
- Límite del multiplicador (Desafío):
- Solicitar e ingresar multiplicadorLimite (por ejemplo, hasta el 10).
- Validación (while): Verificar que multiplicadorLimite > 0.

3. Proceso

Ciclo Externo (for):

- Controla qué tabla se está procesando. Iterar con la variable i desde tablaInicial hasta tablaFinal incrementando de 1 en 1.

Ciclo Interno (for):

- Controla los multiplicadores de la tabla actual. Para cada iteración del ciclo externo, ejecutar un ciclo secundario con la variable j desde 1 hasta multiplicadorLimite incrementando de 1 en 1.

Cálculo:

- En cada paso del ciclo interno, calcular el producto:

4. Salida

- Al inicio del ciclo externo, mostrar el encabezado de la tabla correspondiente:
- "TABLA DEL " + i
- En cada iteración del ciclo interno, imprimir la operación formateada:
i + " x " + j + " = " + (i * j)

5. Fin

- Finalizar la ejecución del algoritmo y liberar los recursos del scanner.

2. Algoritmo (Pseudocódigo en PSeInt)

Algoritmo TablaMultiplicarConfigurable

// Declaración de variables

Definir tablaInicial, tablaFinal, multiplicadorLimite, i, j Como Entero

// Entrada con validación

Escribir "Ingrese la tabla inicial:"

Leer tablaInicial

Escribir "Ingrese la tabla final:"

Leer tablaFinal

// Validar que la tabla inicial no sea mayor que la tabla final

Mientras tablaInicial > tablaFinal Hacer

 Escribir "Error: La tabla inicial no puede ser mayor que la tabla final."

 Escribir "Reingrese la tabla inicial:"

 Leer tablaInicial

 Escribir "Reingrese la tabla final:"

 Leer tablaFinal

FinMientras

// Desafío: Solicitar hasta qué multiplicador generar

Escribir "Ingrese hasta qué multiplicador desea generar las tablas (ej. 10):"

Leer multiplicadorLimite

Mientras multiplicadorLimite <= 0 Hacer

 Escribir "Error: El multiplicador debe ser mayor a 0. Ingrese nuevamente:"

 Leer multiplicadorLimite

FinMientras

// Proceso mediante ciclos 'for' anidados

Para i <- tablaInicial Hasta tablaFinal Con Paso 1 Hacer

 Escribir ""

 Escribir "TABLA DEL ", i

 Para j <- 1 Hasta multiplicadorLimite Con Paso 1 Hacer

 Escribir i, " x ", j, " = ", (i * j)

 FinPara

FinPara

FinAlgoritmo

3. DIAGRAMA DE FLUJO

INICIO



4. Código Java

```
import java.util.Scanner;
```

```
public class TablaMultiplicarConfigurable {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int tablaInicial;
```

```
        int tablaFinal;
```

```
        int multiplicadorLimite;
```

```
        // Entrada y validación del rango de tablas
```

```
        System.out.print("Ingrese la tabla inicial: ");
```

```
        tablaInicial = scanner.nextInt();
```

```
        System.out.print("Ingrese la tabla final: ");
```

```
        tablaFinal = scanner.nextInt();
```

```
        // Validación: tabla inicial no debe ser mayor que tabla final
```

```
        while (tablaInicial > tablaFinal) {
```

```
            System.out.println("Error: La tabla inicial no puede ser mayor que la tabla final.");
```

```
            System.out.print("Ingrese nuevamente la tabla inicial: ");
```

```
            tablaInicial = scanner.nextInt();
```

```
            System.out.print("Ingrese nuevamente la tabla final: ");
```

```
            tablaFinal = scanner.nextInt();
```

```
        }
```

```
        // Desafío: permitir al usuario definir el multiplicador límite
```

```
        System.out.print("Ingrese hasta qué multiplicador desea generar las tablas (ej. 10): ");
```

```
        multiplicadorLimite = scanner.nextInt();
```

```
        while (multiplicadorLimite <= 0) {
```

```
            System.out.print("Error: El multiplicador debe ser un entero positivo. Intente de nuevo: ");
```

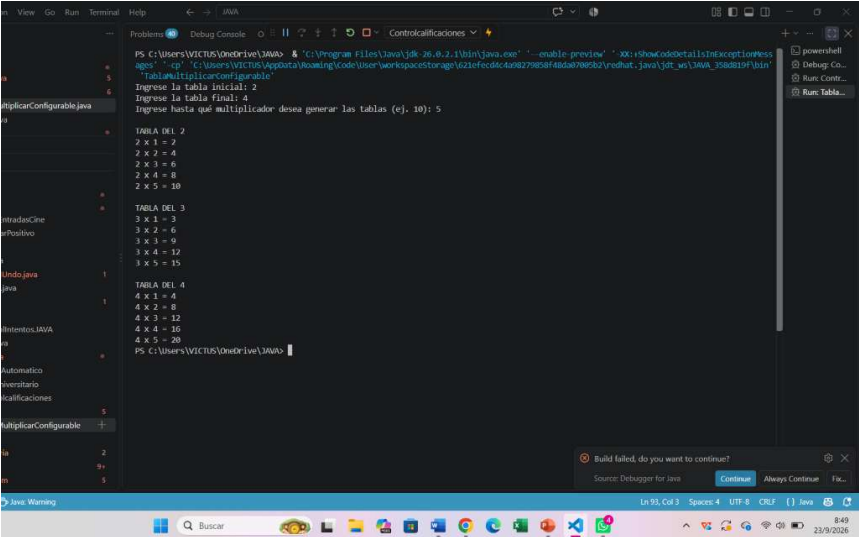
```
            multiplicadorLimite = scanner.nextInt();
```

```
        }
```

```
// Generación de tablas utilizando ciclos 'for' anidados
for (int i = tablaInicial; i <= tablaFinal; i++) {
    System.out.println("\nTABLA DEL " + i);
    for (int j = 1; j <= multiplicadorLimite; j++) {
        System.out.println(i + " x " + j + " = " + (i * j));
    }
}
```

```
scanner.close();
}
```

Run JAVA



5.PRUEBAS DE ESCRITORIO

Prueba de escritorio

tabla inicial	tabla final	multiplicador final	tabla	multiplicador	resultado	salida
3	5	10	3	1	3	3 x 1 = 3
3	5	10	3	2	6	3 x 2 = 6
3	5	10	3	3	9	3 x 3 = 9
3	5	10	3	4	12	3 x 4 = 12
3	5	10	3	5	15	3 x 5 = 15
3	5	10	3	6	18	3 x 6 = 18
3	5	10	3	7	21	3 x 7 = 21
3	5	10	3	8	24	3 x 8 = 24
3	5	10	3	9	27	3 x 9 = 27
3	5	10	3	10	30	3 x 10 = 30
3	5	10	4	1	4	4 x 1 = 4
3	5	10	4	2	8	4 x 2 = 8
3	5	10	4	3	12	4 x 3 = 12
3	5	10	4	4	16	4 x 4 = 16
3	5	10	4	5	20	4 x 5 = 20
3	5	10	4	6	24	4 x 6 = 24
3	5	10	4	7	28	4 x 7 = 28
3	5	10	4	8	32	4 x 8 = 32
3	5	10	4	9	36	4 x 9 = 36
3	5	10	4	10	40	4 x 10 = 40
3	5	10	5	1	5	5 x 1 = 5
3	5	10	5	2	10	5 x 2 = 10
3	5	10	5	3	15	5 x 3 = 15
3	5	10	5	4	20	5 x 4 = 20
3	5	10	5	5	25	5 x 5 = 25
3	5	10	5	6	30	5 x 6 = 30
3	5	10	5	7	35	5 x 7 = 35
3	5	10	5	8	40	5 x 8 = 40
3	5	10	5	9	45	5 x 9 = 45
3	5	10	5	10	50	5 x 10 = 50

EJERCICIO 3. SERIE Y SUMA DE NÚMEROS PARES

1. Análisis: Entrada, Proceso y Salida

Entrada

- N: número entero positivo ingresado por el usuario.

Proceso

- Validar que el número ingresado sea positivo.
- Recorrer los números pares desde 2 hasta N utilizando una estructura for.
- Utilizar un contador para conocer la cantidad de números pares.
- Utilizar un acumulador para obtener la suma de los números pares.
- Calcular el promedio dividiendo la suma para la cantidad de números pares.

Salida

- Serie de números pares desde 2 hasta N.
 - Cantidad de números pares.
 - Suma de los números pares.
 - Promedio de los números pares.
-

2. Pseudocódigo

Inicio

Declarar N, i, cantidad, suma como entero

Declarar promedio como real

Repetir

 Escribir "Ingrese un número entero positivo:"

 Leer N

 Si $N \leq 0$ Entonces

Escribir "Valor inválido. Debe ser positivo."

FinSi

Hasta que $N > 0$

cantidad $\leftarrow 0$

suma $\leftarrow 0$

Escribir "Serie:"

Para $i \leftarrow 2$ Hasta N Con Paso 2 Hacer

Escribir i

cantidad $\leftarrow$ cantidad + 1

suma $\leftarrow$ suma + i

FinPara

Si cantidad > 0 Entonces

promedio $\leftarrow$ suma / cantidad

SiNo

promedio $\leftarrow 0$

FinSi

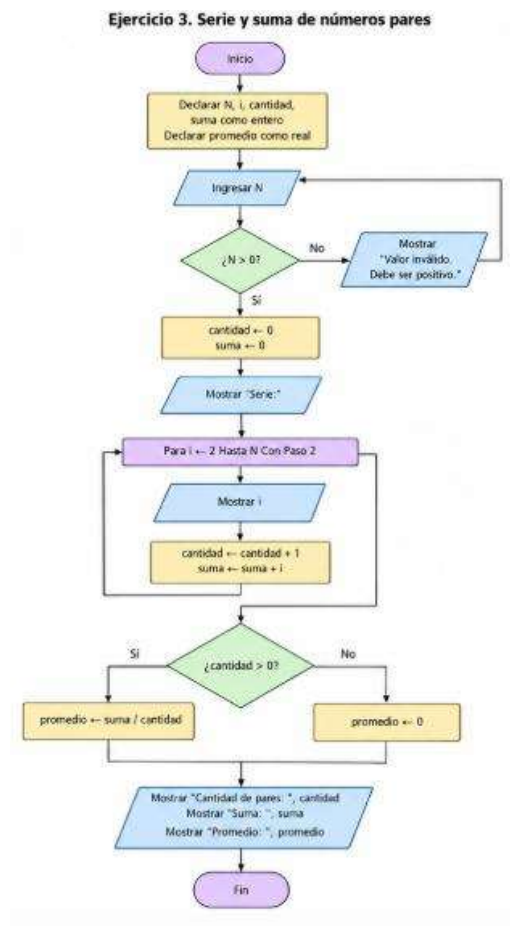
Escribir "Cantidad de pares: ", cantidad

Escribir "Suma: ", suma

Escribir "Promedio: ", promedio

Fin

3. Flujograma en Flowgorithm



4. Código en Java

```
import java.util.Scanner;

public class Ejercicio3Pares {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n;
```

```
int cantidad = 0;

int suma = 0;

double promedio;

do {

    System.out.print("Ingrese un numero entero positivo: ");

    n = sc.nextInt();

    if (n <= 0) {

        System.out.println("Valor invalido. Debe ser positivo.");

    }

} while (n <= 0);

System.out.println("\nSerie:");

for (int i = 2; i <= n; i += 2) {

    System.out.print(i + " ");

    cantidad++;

    suma += i;

}

if (cantidad > 0) {

    promedio = (double) suma / cantidad;

} else {

    promedio = 0;
```

```

    }

    System.out.println("\nCantidad de pares: " + cantidad);

    System.out.println("Suma: " + suma);

    System.out.println("Promedio: " + promedio);

    sc.close();
}
}

```

5. Prueba de escritorio

Datos de entrada

N = 12

i	¿i <= N?	Cantidad	Suma	Salida
2	Sí	1	2	2
4	Sí	2	6	4
6	Sí	3	12	6
8	Sí	4	20	8
10	Sí	5	30	10
12	Sí	6	42	12
14	No	6	42	Termina

Resultado final

Cantidad de pares: 6

Suma: 42

Promedio: 7

Casos límite

N = 1

Cantidad de pares: 0

Suma: 0

Promedio: 0

N = 2

Serie: 2

Cantidad de pares: 1

Suma: 2

Promedio: 2

N = -5

Valor inválido. Debe ser positivo.

El programa vuelve a solicitar el número.

EJERCICIO 4. CAJERO AUTOMÁTICO BÁSICO

1. Análisis: Entrada, Proceso y Salida

Entrada

- Opción seleccionada del menú.
- Monto del depósito.
- Monto del retiro.

Proceso

- Iniciar el saldo con **\$500**.
- Mostrar el menú de opciones.
- Utilizar do-while para mantener activo el programa hasta seleccionar la opción 5.
- Utilizar switch para procesar las opciones.
- Validar que los depósitos no sean negativos.
- Validar que los retiros no sean negativos.
- Validar que el retiro no sea mayor al saldo disponible.
- Utilizar contadores para registrar la cantidad de depósitos y retiros.
- Utilizar acumuladores para registrar el total depositado y retirado.

- Actualizar el saldo después de cada operación válida.

Salida

- Saldo disponible.
 - Mensaje de depósito realizado.
 - Mensaje de retiro realizado.
 - Movimientos realizados.
 - Total depositado.
 - Total retirado.
 - Saldo final.
 - Mensaje de opción inexistente cuando se ingrese una opción que no corresponda al menú.
-

2. Pseudocódigo

Inicio

saldo <- 500

depositosRealizados <- 0

retirosRealizados <- 0

totalDepositado <- 0

totalRetirado <- 0

Repetir

Mostrar "CAJERO AUTOMÁTICO"

Mostrar "1. Consultar saldo"

Mostrar "2. Depositar"

Mostrar "3. Retirar"

Mostrar "4. Mostrar movimientos"

Mostrar "5. Salir"

Leer opcion

Según opcion Hacer

Caso 1:

Mostrar saldo

Caso 2:

Mostrar "Ingrese el valor a depositar"

Leer monto

Si monto < 0 Entonces

Mostrar "No se permiten depósitos negativos"

SiNo

saldo <- saldo + monto

depositosRealizados <- depositosRealizados + 1

totalDepositado <- totalDepositado + monto

Mostrar "Depósito realizado correctamente"

FinSi

Caso 3:

Mostrar "Ingrese el valor a retirar"

Leer monto

Si monto < 0 Entonces

Mostrar "No se permiten retiros negativos"

SiNo Si monto > saldo Entonces

Mostrar "No puede retirar más dinero del disponible"

SiNo

saldo <- saldo - monto

retirosRealizados <- retirosRealizados + 1

totalRetirado <- totalRetirado + monto

Mostrar "Retiro realizado correctamente"

FinSi

Caso 4:

Mostrar "Depósitos realizados"

Mostrar "Retiros realizados"

Mostrar "Total depositado"

Mostrar "Total retirado"

Mostrar "Saldo actual"

Caso 5:

Mostrar "Saliendo del cajero"

Otro:

Mostrar "Opción inexistente"

FinSegún

Hasta que opcion = 5

Mostrar "RESUMEN FINAL"

Mostrar "Depósitos realizados"

Mostrar "Retiros realizados"

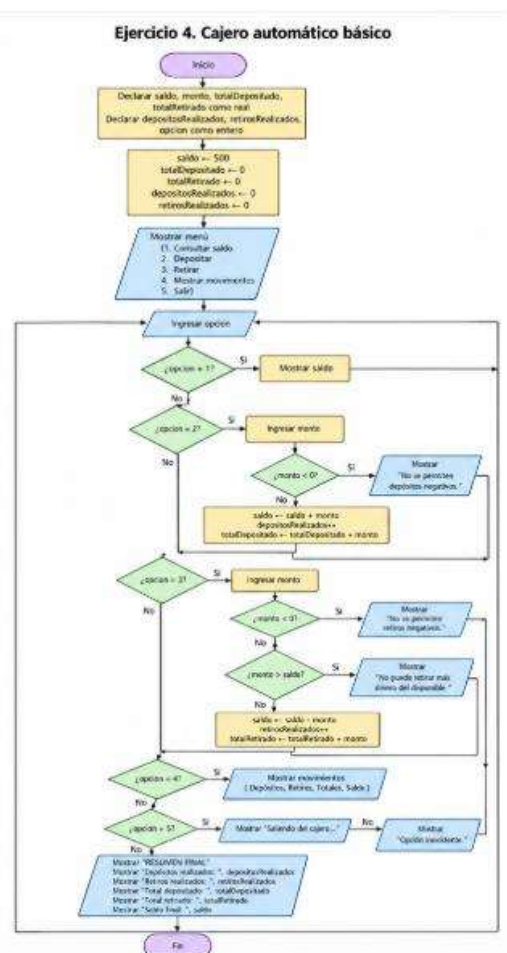
Mostrar "Total depositado"

Mostrar "Total retirado"

Mostrar "Saldo final"

Fin

3. Flujograma en Flowgorithm



4. Código en Java

```
import java.util.Scanner;
```

```
public class Ejercicio4Cajero {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        double saldo = 500;

        double monto;

        double totalDepositado = 0;

        double totalRetirado = 0;

        int depositosRealizados = 0;

        int retirosRealizados = 0;

        int opcion;

        do {

            System.out.println("\n=====");
            System.out.println("  CAJERO AUTOMATICO");
            System.out.println("=====");
            System.out.println("1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("4. Mostrar movimientos");
            System.out.println("5. Salir");
```

```
System.out.print("Seleccione una opcion: ");
```

```
opcion = sc.nextInt();
```

```
switch (opcion) {
```

```
    case 1:
```

```
        System.out.printf("Saldo disponible: $%.2f%n", saldo);
```

```
        break;
```

```
    case 2:
```

```
        System.out.print("Ingrese el valor a depositar: ");
```

```
        monto = sc.nextDouble();
```

```
        if (monto < 0) {
```

```
            System.out.println(
```

```
                "No se permiten depositos negativos."
```

```
            );
```

```
        } else {
```

```
            saldo += monto;
```

```
depositosRealizados++;
```

```
totalDepositado += monto;
```

```
System.out.println(
```

```
    "Deposito realizado correctamente."
```

```
);
```

```
}
```

```
break;
```

```
case 3:
```

```
System.out.print("Ingrese el valor a retirar: ");
```

```
monto = sc.nextDouble();
```

```
if (monto < 0) {
```

```
    System.out.println(
```

```
        "No se permiten retiros negativos."
```

```
);
```

```
} else if (monto > saldo) {
```

```
    System.out.println(
```

```
        "No puede retirar mas dinero del disponible."
```

```
);
```

```
} else {  
  
    saldo -= monto;  
  
    retirosRealizados++;  
  
    totalRetirado += monto;  
  
    System.out.println(  
        "Retiro realizado correctamente."  
    );  
}
```

```
break;
```

```
case 4:
```

```
System.out.println("\n--- MOVIMIENTOS ---");
```

```
System.out.println(  
    "Depositos realizados: "  
    + depositosRealizados  
);
```

```
System.out.println(  
    "Retiros realizados: "
```

```
        + retirosRealizados  
    );
```

```
System.out.printf(  
    "Total depositado: $%.2f%n",  
    totalDepositado  
);
```

```
System.out.printf(  
    "Total retirado: $%.2f%n",  
    totalRetirado  
);
```

```
System.out.printf(  
    "Saldo actual: $%.2f%n",  
    saldo  
);
```

```
break;
```

```
case 5:
```

```
System.out.println(  
    "Saliendo del cajero..."  
);
```

```
break;
```

default:

```
        System.out.println(  
            "Opcion inexistente."  
        );  
    }
```

```
} while (opcion != 5);
```

```
System.out.println("\n===== RESUMEN FINAL =====");
```

```
System.out.println(  
    "Depositos realizados: "  
    + depositosRealizados  
);
```

```
System.out.println(  
    "Retiros realizados: "  
    + retirosRealizados  
);
```

```
System.out.printf(  
    "Total depositado: $%.2f%n",  
    totalDepositado  
);
```

```
System.out.printf(
    "Total retirado: $%.2f%n",
    totalRetirado
);

System.out.printf(
    "Saldo final: $%.2f%n",
    saldo
);

sc.close();
}
}
```

5. Prueba de escritorio

Datos iniciales

Saldo inicial = \$500

Depósitos realizados = 0

Retiros realizados = 0

Total depositado = \$0

Total retirado = \$0

Prueba principal

Se realizan las siguientes operaciones:

1. Depositar \$100.
2. Retirar \$50.
3. Mostrar movimientos.
4. Salir.

Paso	Opción	Monto	Saldo	Depósitos	Retiros	Total depositado	Total retirado
Inicio	-	-	\$500	0	0	\$0	\$0
1	2	\$100	\$600	1	0	\$100	\$0
2	3	\$50	\$550	1	1	\$100	\$50
3	4	-	\$550	1	1	\$100	\$50
4	5	-	\$550	1	1	\$100	\$50

Resultado final

Depósitos realizados: 1

Retiros realizados: 1

Total depositado: \$100.00

Total retirado: \$50.00

Saldo final: \$550.00

Pruebas de validación

Depósito negativo

Opción: 2

Monto: -20

Resultado:

No se permiten depositos negativos.

El saldo permanece en \$500.

Retiro negativo

Opción: 3

Monto: -10

Resultado:

No se permiten retiros negativos.

El saldo no cambia.

Retiro mayor al saldo

Saldo: \$500

Opción: 3

Monto: \$600

Resultado:

No puede retirar mas dinero del disponible.

El saldo permanece en \$500.

Opción inexistente

Opción: 8

Resultado:

Opcion inexistente.

El programa vuelve a mostrar el menú.

EJERCICIO 5. ESTACIONAMIENTO UNIVERSITARIO

1. ENUNCIADO

Desarrolle un programa para registrar los vehículos que utilizan un estacionamiento. Por cada vehículo se solicita el tipo y las horas estacionado. Las tarifas son: motocicleta \$0.50 por hora, automóvil \$1.00 por hora y camioneta \$1.50 por hora. El programa permite registrar varios vehículos hasta que el usuario decida terminar. Al final muestra la cantidad de motocicletas, automóviles, camionetas, total de vehículos, total recaudado y promedio pagado. Las horas deben ser mayores que cero.

2. ANÁLISIS

Entrada	Proceso	Salida
Tipo de vehículo, horas estacionado y decisión de continuar o terminar.	Validar el tipo y las horas; seleccionar la tarifa con switch; calcular el pago; actualizar contadores y acumuladores; calcular el promedio al finalizar.	Cantidad de motocicletas, automóviles, camionetas, total de vehículos, total recaudado y promedio pagado.

3. ALGORITMO

1. Inicializar los contadores de cada tipo de vehículo, el total de vehículos y el total recaudado en cero.
2. Solicitar el tipo de vehículo.
3. Validar que el tipo sea 1, 2 o 3.
4. Seleccionar la tarifa correspondiente mediante switch.
5. Solicitar las horas estacionado y validar que sean mayores que cero.
6. Calcular el pago multiplicando las horas por la tarifa.
7. Incrementar el contador correspondiente y acumular el pago.
8. Preguntar si se desea registrar otro vehículo.
9. Repetir el proceso mientras la respuesta sea S.
10. Calcular el promedio pagado dividiendo el total recaudado para el total de vehículos.
11. Mostrar el reporte final.

4. PSEUDOCÓDIGO

```
Inicio
    motocicletas <- 0
    automoviles <- 0
    camionetas <- 0
    totalVehiculos <- 0
    totalRecaudado <- 0

    Repetir
        Repetir
            Escribir "1. Motocicleta  2. Automóvil  3. Camioneta"
            Leer tipo
            Hasta que tipo >= 1 Y tipo <= 3
```

```

Segun tipo Hacer
  1:
    tarifa <- 0.50
    motocicletas <- motocicletas + 1
  2:
    tarifa <- 1.00
    automoviles <- automoviles + 1
  3:
    tarifa <- 1.50
    camionetas <- camionetas + 1
FinSegun

Repetir
  Escribir "Horas estacionado:"
  Leer horas
Hasta que horas > 0

pago <- horas * tarifa
totalVehiculos <- totalVehiculos + 1
totalRecaudado <- totalRecaudado + pago

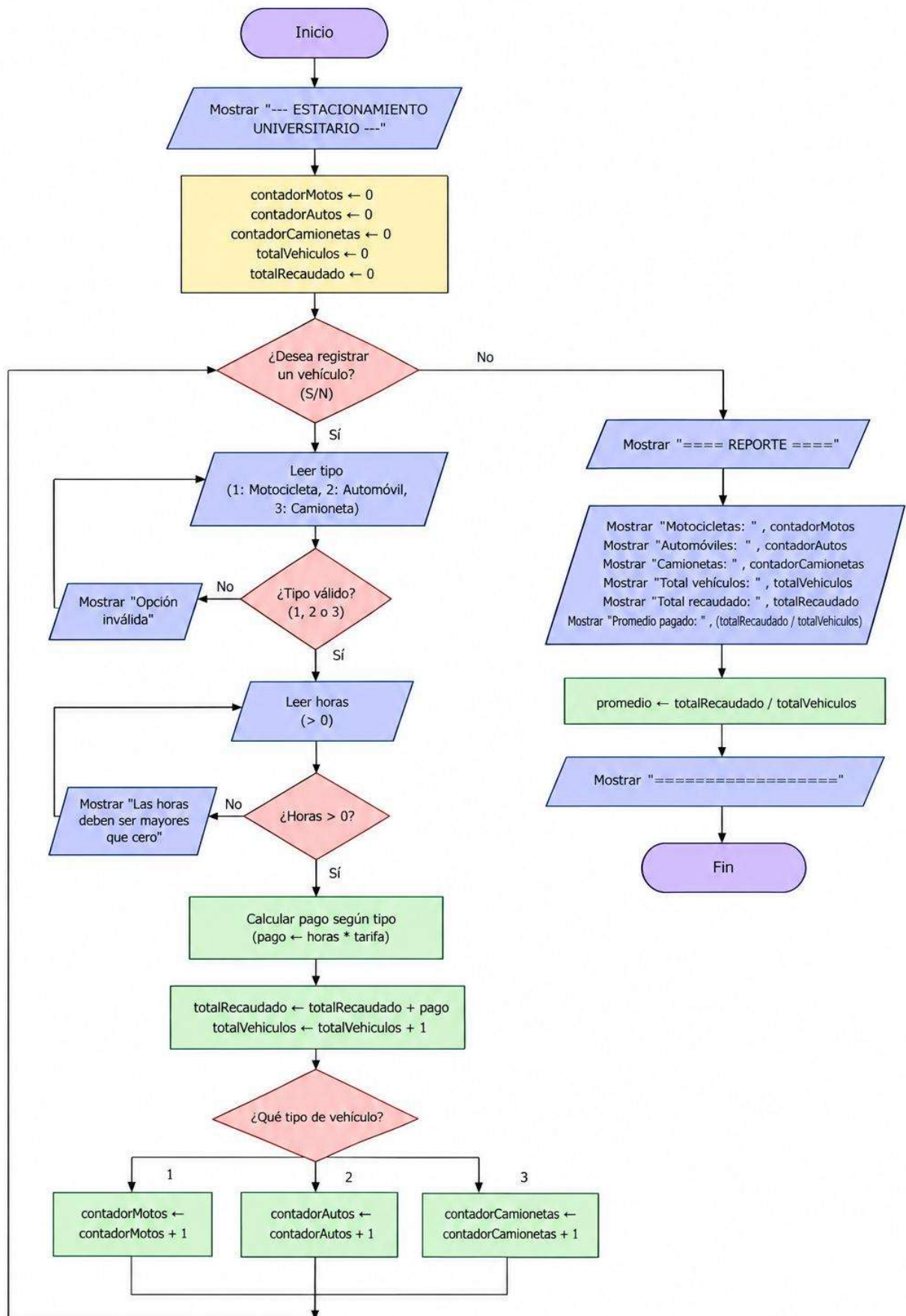
  Escribir "¿Desea registrar otro vehículo? (S/N)"
  Leer continuar
Hasta que continuar = "N"

promedio <- totalRecaudado / totalVehiculos

Escribir "===== REPORTE ====="
Escribir "Motocicletas: ", motocicletas
Escribir "Automóviles: ", automoviles
Escribir "Camionetas: ", camionetas
Escribir "Total vehículos: ", totalVehiculos
Escribir "Total recaudado: ", totalRecaudado
Escribir "Promedio pagado: ", promedio
Fin

```

5. DIAGRAMA DE FLUJO (FLOWGORITHM)



6. CÓDIGO EN JAVA

```
import java.util.Scanner;

public class EstacionamientoUniversitario {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int motocicletas = 0, automoviles = 0, camionetas = 0;
        int totalVehiculos = 0;
        double totalRecaudado = 0;
        String continuar;

        do {
            int tipo;
            do {
                System.out.println("\n1. Motocicleta");
                System.out.println("2. Automóvil");
                System.out.println("3. Camioneta");
                System.out.print("Seleccione el tipo: ");
                while (!sc.hasNextInt()) {
                    System.out.print("Ingrese un número válido: ");
                    sc.next();
                }
                tipo = sc.nextInt();
                if (tipo < 1 || tipo > 3) {
                    System.out.println("Tipo no válido.");
                }
            } while (tipo < 1 || tipo > 3);

            double tarifa = 0;

            switch (tipo) {
                case 1:
                    tarifa = 0.50;
                    motocicletas++;
                    break;
                case 2:
                    tarifa = 1.00;
                    automoviles++;
                    break;
                case 3:
                    tarifa = 1.50;
                    camionetas++;
                    break;
            }

            double horas;
            do {
                System.out.print("Horas estacionado: ");
                while (!sc.hasNextDouble()) {
                    System.out.print("Ingrese un número válido: ");
                    sc.next();
                }
                horas = sc.nextDouble();
                if (horas <= 0) {
```

```

        System.out.println("Las horas deben ser mayores que cero.");
    }
} while (horas <= 0);

double pago = horas * tarifa;
totalVehiculos++;
totalRecaudado += pago;

do {
    System.out.print("¿Desea registrar otro vehículo? (S/N): ");
    continuar = sc.next().toUpperCase();
} while (!continuar.equals("S") && !continuar.equals("N"));

} while (continuar.equals("S"));

double promedio = totalRecaudado / totalVehiculos;

System.out.println("\n===== REPORTE =====");
System.out.println("Motocicletas: " + motocicletas);
System.out.println("Automóviles: " + automoviles);
System.out.println("Camionetas: " + camionetas);
System.out.println("Total vehículos: " + totalVehiculos);
System.out.printf("Total recaudado: $%.2f%n", totalRecaudado);
System.out.printf("Promedio pagado: $%.2f%n", promedio);
System.out.println("=====");
sc.close();
}
}

```

7. PRUEBA DE ESCRITORIO

Prueba	Datos de entrada	Resultado esperado
1	Motocicleta, 2 horas, N	1 motocicleta; total \$1.00; promedio \$1.00
2	Automóvil, 3 horas; Camioneta, 2 horas; N	2 vehículos; total \$7.50; promedio \$3.75
3	Motocicleta, 1.5 horas; Automóvil, 2 horas; Camioneta, 4 horas; N	3 vehículos; total \$8.50; promedio \$2.83
4	Horas = -2, luego 2 horas; Motocicleta; N	Se rechaza -2; pago \$1.00; 1 vehículo

EJERCICIO 6. TRIÁNGULO Y PATRONES

1. ENUNCIADO

Solicite un número entre 2 y 10. Con ese número se debe mostrar primero un triángulo creciente de asteriscos, después un triángulo decreciente y finalmente un patrón numérico donde cada fila contiene los números desde 1 hasta el número de fila. Se debe explicar mediante una tabla de trazas cómo interactúan el ciclo exterior y el ciclo interior.

2. ANÁLISIS

Entrada	Proceso	Salida
Un número entero n entre 2 y 10.	Validar n; usar un ciclo for exterior para controlar las filas y un ciclo for interior para controlar cuántos símbolos o números se imprimen en cada fila.	Tres patrones: asteriscos crecientes, asteriscos decrecientes y números crecientes.

3. ALGORITMO

1. Solicitar n.
2. Validar que n esté entre 2 y 10.
3. Usar un for exterior desde 1 hasta n.
4. Usar un for interior desde 1 hasta i e imprimir un asterisco.
5. Pasar a la siguiente línea.
6. Usar un for exterior desde n hasta 1.
7. Usar un for interior desde 1 hasta i e imprimir un asterisco.
8. Pasar a la siguiente línea.
9. Usar un for exterior desde 1 hasta n.
10. Usar un for interior desde 1 hasta i e imprimir el valor de j.
11. Pasar a la siguiente línea.

4. PSEUDOCÓDIGO

```
Inicio
  Repetir
    Escribir "Ingrese un número entre 2 y 10:"
    Leer n
    Hasta que n >= 2 Y n <= 10

    Escribir "Triángulo creciente"
    Para i <- 1 Hasta n Hacer
      Para j <- 1 Hasta i Hacer
        Escribir "*" Sin Saltar
      FinPara
      Escribir ""
    FinPara

    Escribir "Triángulo decreciente"
```

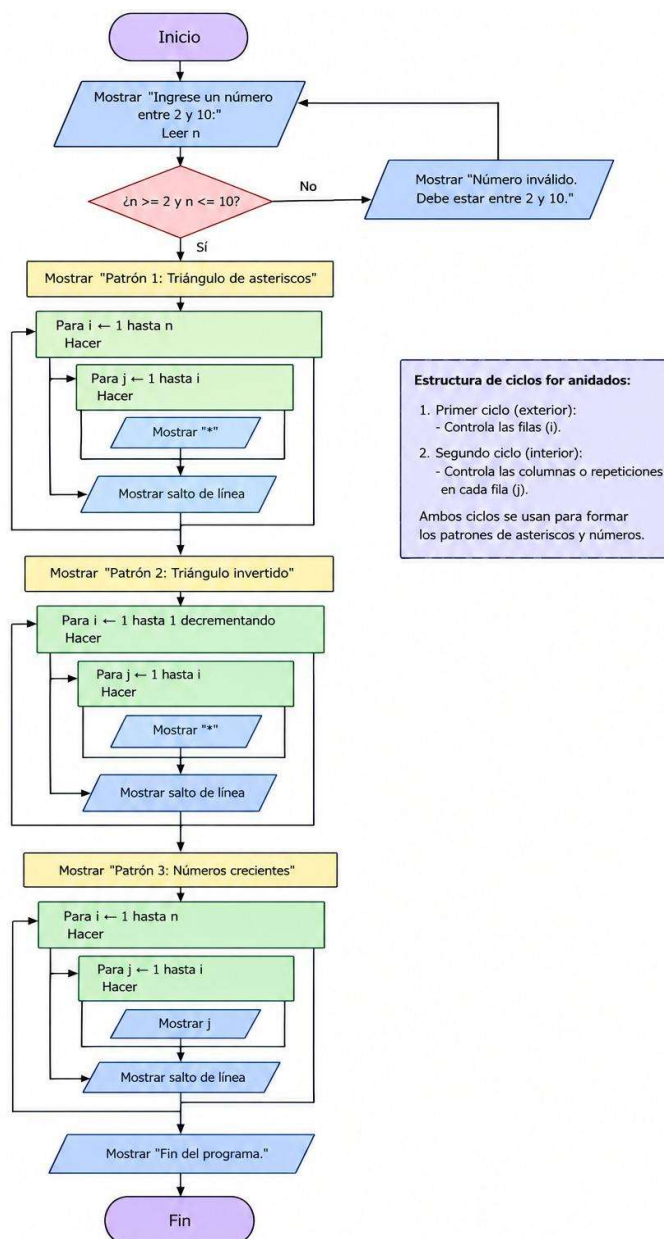
```

Para i <- n Hasta 1 Con Paso -1 Hacer
  Para j <- 1 Hasta i Hacer
    Escribir "*" Sin Saltar
  FinPara
  Escribir ""
FinPara

Escribir "Patrón numérico"
Para i <- 1 Hasta n Hacer
  Para j <- 1 Hasta i Hacer
    Escribir j Sin Saltar
  FinPara
  Escribir ""
FinPara
Fin

```

5. DIAGRAMA DE FLUJO (FLOWGORITHM)



La tabla de trazas muestra cómo el ciclo exterior (i) determina cuántas veces se repite el ciclo interior (j).

i	j (valores recorridos)	Cantidad de símbolos
i (ciclo exterior)	j (ciclo interior)	Salida de la fila
1	1	*
2	1, 2	**
3	1, 2, 3	***
4	1, 2, 3, 4	****
5	1, 2, 3, 4, 5	*****

6. CÓDIGO EN JAVA

```
import java.util.Scanner;

public class TrianguloYPatrones {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;

        do {
            System.out.print("Ingrese un número entre 2 y 10: ");
            while (!sc.hasNextInt()) {
                System.out.print("Ingrese un número entero válido: ");
                sc.next();
            }
            n = sc.nextInt();

            if (n < 2 || n > 10) {
                System.out.println("El número debe estar entre 2 y 10.");
            }
        } while (n < 2 || n > 10);

        System.out.println("\nTriángulo creciente:");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nTriángulo decreciente:");
        for (int i = n; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nPatrón numérico:");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

```
        sc.close();  
    }  
}
```

7. PRUEBA DE ESCRITORIO

Prueba	Entrada	Resultado esperado
1	n = 2	Patrones con 2 filas: *, ** / **, * / 1, 12
2	n = 3	Patrones con 3 filas correctamente formados
3	n = 5	Patrones con 5 filas, igual al ejemplo del enunciado
4	n = 11 y luego n = 4	11 se rechaza; con 4 se muestran los tres patrones

EJERCICIO 7. CONTROL DE VENTAS DE CAFETERÍA

1. ENUNCIADO

Construya un sistema para controlar las ventas de una cafetería universitaria. El menú debe permitir registrar una venta, mostrar estadísticas, mostrar la tabla de productos y salir. Los productos son: Café \$1.00, Sándwich \$2.50, Jugo \$1.50 y Empanada \$1.25. Al registrar una venta se solicita el producto y la cantidad. El reporte muestra el número de ventas, cantidad total de productos, total recaudado, promedio por venta y producto con mayor cantidad vendida. Todas las entradas deben ser validadas.

2. ANÁLISIS

Entrada	Proceso	Salida
Opción del menú, producto y cantidad vendida.	Validar entradas; seleccionar producto y precio con switch; calcular subtotal; actualizar contadores y acumuladores; comparar cantidades para determinar el producto con mayor venta; calcular promedio por venta.	Estadísticas de ventas, tabla de productos y producto con mayor cantidad vendida.

3. ALGORITMO

1. Inicializar contadores de ventas y cantidades por producto y el total recaudado.
2. Mostrar el menú principal.
3. Validar la opción seleccionada.
4. Si es 1, solicitar producto y cantidad, validarlos y calcular el subtotal.
5. Actualizar la cantidad del producto, número de ventas, cantidad total y total recaudado.
6. Si es 2, mostrar estadísticas y calcular el promedio por venta.
7. Si es 3, mostrar la tabla de productos y precios.
8. Si es 4, finalizar el programa.
9. Repetir el menú mientras la opción sea diferente de 4.

4. PSEUDOCÓDIGO

```
Inicio
    ventas <- 0
    cafe <- 0
    sandwich <- 0
    jugo <- 0
    empanada <- 0
    cantidadTotal <- 0
    totalRecaudado <- 0

    Repetir
        Mostrar menú
        Leer opcion

        Segun opcion Hacer
            1:
                Repetir
```

```

        Leer producto
Hasta que producto >= 1 Y producto <= 4

Repetir
    Leer cantidad
Hasta que cantidad > 0

Segun producto Hacer
    1:
        precio <- 1.00
        cafe <- cafe + cantidad
    2:
        precio <- 2.50
        sandwich <- sandwich + cantidad
    3:
        precio <- 1.50
        jugo <- jugo + cantidad
    4:
        precio <- 1.25
        empanada <- empanada + cantidad
FinSegun

subtotal <- precio * cantidad
ventas <- ventas + 1
cantidadTotal <- cantidadTotal + cantidad
totalRecaudado <- totalRecaudado + subtotal

2:
Si ventas > 0 Entonces
    promedio <- totalRecaudado / ventas
    Determinar el producto con mayor cantidad vendida
    Mostrar estadísticas
SiNo
    Mostrar "No existen ventas"
FinSi

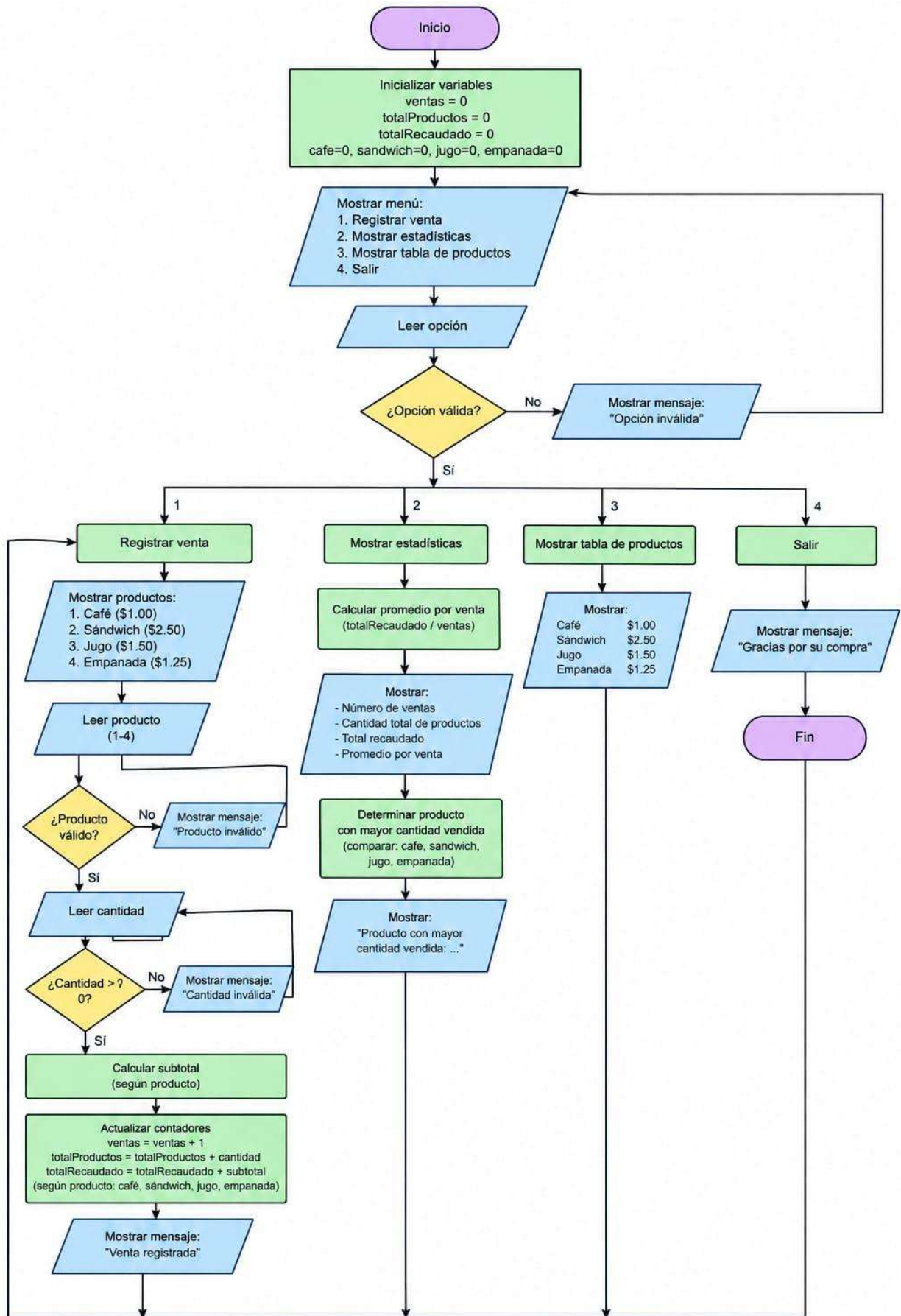
3:
Mostrar tabla de productos y precios

4:
Mostrar "Programa finalizado"

De Otro Modo:
    Mostrar "Opción no válida"
FinSegun
Hasta que opcion = 4
Fin

```

5. DIAGRAMA DE FLUJO (FLOWGORITHM)



6. CÓDIGO EN JAVA

```
import java.util.Scanner;

public class ControlVentasCafeteria {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int ventas = 0;
        int cafe = 0, sandwich = 0, jugo = 0, empanada = 0;
        int cantidadTotal = 0;
        double totalRecaudado = 0;
        int opcion;

        do {
            System.out.println("\n=====");
            System.out.println("      CAFETERÍA UNIVERSITARIA");
            System.out.println("=====");
            System.out.println("1. Registrar venta");
            System.out.println("2. Mostrar estadísticas");
            System.out.println("3. Mostrar tabla de productos");
            System.out.println("4. Salir");
            System.out.println("=====");
            System.out.print("Seleccione una opción: ");

            while (!sc.hasNextInt()) {
                System.out.print("Ingrese una opción válida: ");
                sc.next();
            }
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    int producto;
                    do {
                        System.out.println("\n1. Café - $1.00");
                        System.out.println("2. Sándwich - $2.50");
                        System.out.println("3. Jugo - $1.50");
                        System.out.println("4. Empanada - $1.25");
                        System.out.print("Seleccione el producto: ");
                        while (!sc.hasNextInt()) {
                            System.out.print("Ingrese un número válido: ");
                            sc.next();
                        }
                        producto = sc.nextInt();
                    } while (producto < 1 || producto > 4);

                    int cantidad;
                    do {
                        System.out.print("Ingrese la cantidad: ");
                        while (!sc.hasNextInt()) {
                            System.out.print("Ingrese una cantidad válida: ");
                            sc.next();
                        }
                    }
                    cantidad = sc.nextInt();
                    if (cantidad <= 0) {
```

```

        System.out.println("La cantidad debe ser mayor que
cero.");
    }
} while (cantidad <= 0);

double precio = 0;

switch (producto) {
    case 1:
        precio = 1.00;
        cafe += cantidad;
        break;
    case 2:
        precio = 2.50;
        sandwich += cantidad;
        break;
    case 3:
        precio = 1.50;
        jugo += cantidad;
        break;
    case 4:
        precio = 1.25;
        empanada += cantidad;
        break;
}

double subtotal = precio * cantidad;
ventas++;
cantidadTotal += cantidad;
totalRecaudado += subtotal;

System.out.printf("Venta registrada. Subtotal: $%.2f%n",
subtotal);

break;

case 2:
    System.out.println("\n===== ESTADÍSTICAS =====");
    if (ventas == 0) {
        System.out.println("No existen ventas registradas.");
    } else {
        double promedio = totalRecaudado / ventas;
        String mayorProducto = "Café";
        int mayorCantidad = cafe;

        if (sandwich > mayorCantidad) {
            mayorCantidad = sandwich;
            mayorProducto = "Sándwich";
        }
        if (jugo > mayorCantidad) {
            mayorCantidad = jugo;
            mayorProducto = "Jugo";
        }
        if (empanada > mayorCantidad) {
            mayorCantidad = empanada;
            mayorProducto = "Empanada";
        }
    }
}

```

```

    }

    System.out.println("Número de ventas: " + ventas);
    System.out.println("Cantidad total de productos: " +
cantidadTotal);

    System.out.printf("Total recaudado: $%.2f%n",
totalRecaudado);

    System.out.printf("Promedio por venta: $%.2f%n", promedio);
    System.out.println("Producto con mayor cantidad vendida: "
        + mayorProducto + " (" + mayorCantidad + ")");
    }
    break;

case 3:
    System.out.println("\n===== TABLA DE PRODUCTOS =====");
    System.out.println("Café          $1.00");
    System.out.println("Sándwich    $2.50");
    System.out.println("Jugo        $1.50");
    System.out.println("Empanada    $1.25");
    break;

case 4:
    System.out.println("Programa finalizado.");
    break;

default:
    System.out.println("Opción no válida.");
}

} while (opcion != 4);

sc.close();
}
}

```

7. PRUEBA DE ESCRITORIO

Prueba	Datos de entrada	Resultado esperado
1	Venta: Café, cantidad 2; luego estadísticas	1 venta; 2 productos; \$2.00; promedio \$2.00; mayor: Café (2)
2	Venta: Sándwich, 2; Jugo, 3; estadísticas	2 ventas; 5 productos; \$9.50; promedio \$4.75; mayor: Jugo (3)
3	Venta: Empanada, 4; Café, 1; Sándwich, 2; estadísticas	3 ventas; 7 productos; \$10.25; promedio \$3.42; mayor: Empanada (4)
4	Cantidad = -2 y luego 3 cafés; estadísticas	-2 se rechaza; 1 venta; 3 productos; \$3.00; mayor: Café (3)

EJERCICIOS 8, 9 Y 10 – ANÁLISIS, ALGORITMO, DIAGRAMAS, JAVA Y PRUEBAS

Programación en Java

EJERCICIO 8. Detector y depurador de errores

Problema original: el programa inicia numero en 1 y repite mientras numero $\leq$ 10, pero nunca modifica numero. Por eso la condición siempre permanece verdadera y el ciclo es infinito.

1. Análisis del problema

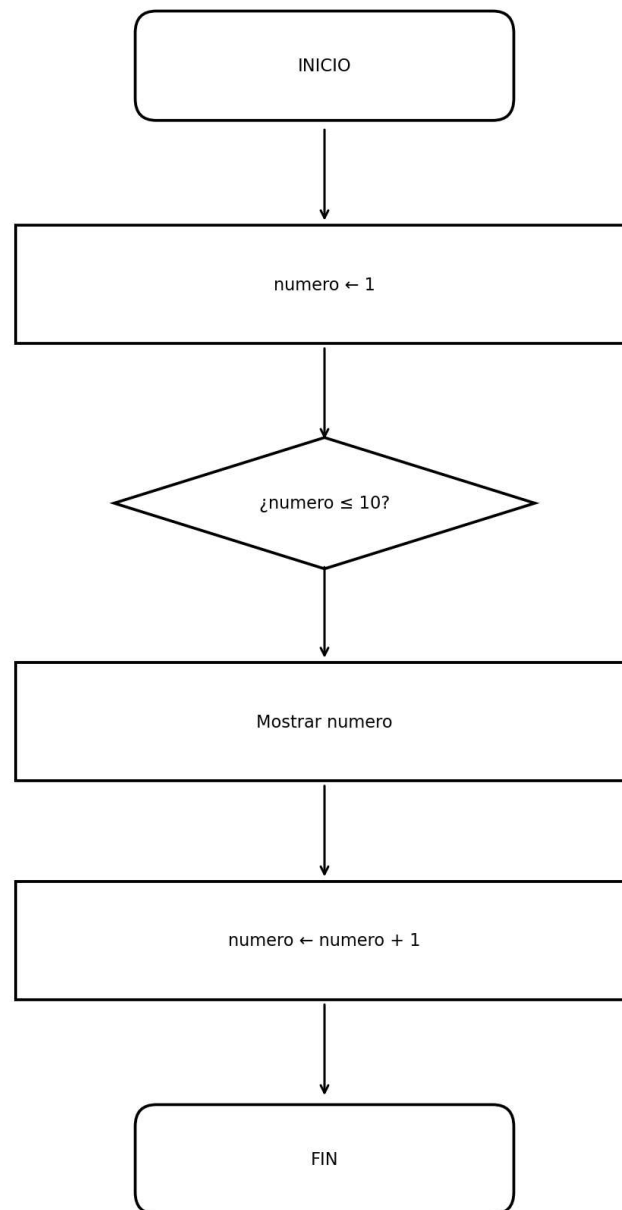
Inicio	Inicializar la variable numero en 1.
Entrada	No requiere datos ingresados por el usuario.
Proceso	Mientras numero sea menor o igual que 10, mostrarlo y aumentar numero en 1.
Salida	Los números del 1 al 10, uno por línea.
Fin	Terminar cuando numero llegue a 11.

2. Algoritmo

```
Inicio
    numero  $\leftarrow$  1
    Mientras numero  $\leq$  10 hacer
        Mostrar numero
        numero  $\leftarrow$  numero + 1
    Fin Mientras
Fin
```

3. Diagrama de flujo

Ejercicio 8 – Ciclo while corregido



Nota: desde la decisión “¿numero ≤ 10?” se regresa al ciclo después de incrementar numero.

4. Código Java corregido

```
public class Ejercicio8 {  
    public static void main(String[] args) {  
        int numero = 1;  
  
        while (numero <= 10) {  
            System.out.println(numero);  
            numero++;  
        }  
    }  
}
```

```
}  
}  
}
```

5. Prueba de escritorio

Iteración	numero antes	¿numero ≤ 10?	Salida / cambio
1	1	Sí	Imprime 1; numero pasa a 2
2	2	Sí	Imprime 2; numero pasa a 3
3	3	Sí	Imprime 3; numero pasa a 4
4	4	Sí	Imprime 4; numero pasa a 5
5	5	Sí	Imprime 5; numero pasa a 6
6	6	Sí	Imprime 6; numero pasa a 7
7	7	Sí	Imprime 7; numero pasa a 8
8	8	Sí	Imprime 8; numero pasa a 9
9	9	Sí	Imprime 9; numero pasa a 10
10	10	Sí	Imprime 10; numero pasa a 11
11	11	No	Termina el ciclo

Resultado esperado: 1, 2, 3, 4, 5, 6, 7, 8, 9 y 10.

6. Documentación del error

- ERROR ENCONTRADO: falta incrementar la variable numero dentro del while.
- CAUSA: la condición numero <= 10 nunca cambia porque numero conserva el valor 1.
- SOLUCIÓN: agregar numero++ dentro del ciclo.
- RESULTADO: el programa imprime del 1 al 10 y finaliza correctamente.

EJERCICIO 9. Estadísticas de una encuesta universitaria

El programa solicita primero la cantidad de estudiantes. Para cada estudiante valida edad, semestre y horas de estudio. Luego calcula promedios, identifica al estudiante con más horas, cuenta quienes estudian menos de 2 horas y obtiene la cantidad de estudiantes por semestre mediante ciclos anidados.

1. Análisis del problema

Inicio	Solicitar la cantidad de estudiantes y preparar acumuladores y contadores.
Entrada	Cantidad de estudiantes, edad (16–80), semestre (1–10) y horas de estudio por día (0–24).
Proceso	Validar datos; acumular edades y horas; buscar mayor cantidad de horas; contar menos de 2 horas; contar estudiantes por semestre con ciclos anidados.
Salida	Edad promedio, horas promedio, estudiante con mayor cantidad de horas, cantidad con menos de 2 horas y cantidad por semestre.
Fin	Mostrar todas las estadísticas y finalizar.

2. Algoritmo

```
Inicio
  Leer n
  Crear arreglos edad[n], semestre[n], horas[n]
  sumaEdad ← 0
  sumaHoras ← 0
  menor2 ← 0
  mayorHoras ← -1
  estudianteMayor ← 0

  Para i ← 0 hasta n-1
    Leer y validar edad[i] entre 16 y 80
    Leer y validar semestre[i] entre 1 y 10
    Leer y validar horas[i] entre 0 y 24

    sumaEdad ← sumaEdad + edad[i]
    sumaHoras ← sumaHoras + horas[i]

    Si horas[i] > mayorHoras entonces
      mayorHoras ← horas[i]
      estudianteMayor ← i + 1
    Fin Si

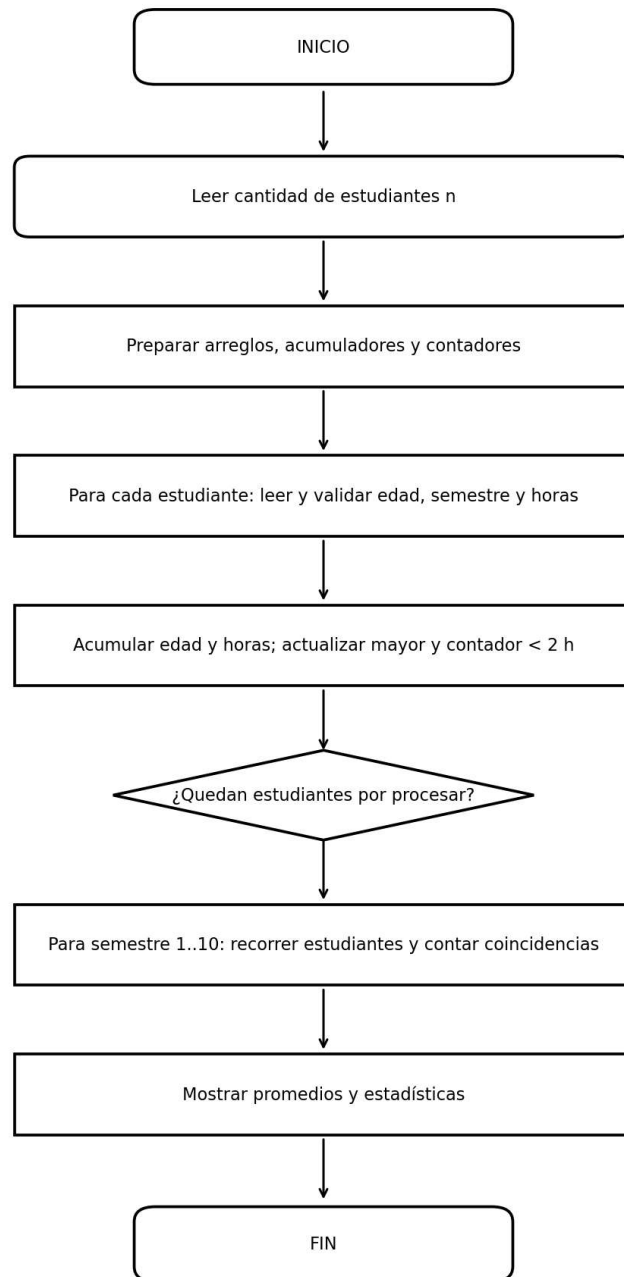
    Si horas[i] < 2 entonces
      menor2 ← menor2 + 1
    Fin Si
  Fin Para

  Para semestreActual ← 1 hasta 10
    contador ← 0
    Para i ← 0 hasta n-1
      Si semestre[i] = semestreActual entonces
        contador ← contador + 1
      Fin Si
    Fin Para
    Mostrar semestreActual y contador
  Fin Para

  Mostrar promedios y estadísticas
Fin
```

3. Diagrama de flujo

Ejercicio 9 – Encuesta universitaria



4. Código Java

```
import java.util.Scanner;

public class Ejercicio9 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n;
        do {
            System.out.print("Cantidad de estudiantes: ");
```

```

        n = sc.nextInt();
        if (n <= 0) System.out.println("Debe ser mayor que 0.");
    } while (n <= 0);

    int[] edad = new int[n];
    int[] semestre = new int[n];
    double[] horas = new double[n];

    int sumaEdad = 0;
    double sumaHoras = 0;
    int menor2 = 0;
    double mayorHoras = -1;
    int estudianteMayor = 0;

    for (int i = 0; i < n; i++) {
        do {
            System.out.print("Edad del estudiante " + (i + 1) + ": ");
            edad[i] = sc.nextInt();
        } while (edad[i] < 16 || edad[i] > 80);

        do {
            System.out.print("Semestre (1-10): ");
            semestre[i] = sc.nextInt();
        } while (semestre[i] < 1 || semestre[i] > 10);

        do {
            System.out.print("Horas de estudio por día (0-24): ");
            horas[i] = sc.nextDouble();
        } while (horas[i] < 0 || horas[i] > 24);

        sumaEdad += edad[i];
        sumaHoras += horas[i];

        if (horas[i] > mayorHoras) {
            mayorHoras = horas[i];
            estudianteMayor = i + 1;
        }

        if (horas[i] < 2) {
            menor2++;
        }
    }

    System.out.println("\n--- ESTADISTICAS ---");
    System.out.printf("Edad promedio: %.2f%n", (double) sumaEdad / n);
    System.out.printf("Horas promedio: %.2f%n", sumaHoras / n);
    System.out.println("Estudiante con mayor cantidad de horas: "
        + estudianteMayor + " (" + mayorHoras + " h)");
    System.out.println("Estudiantes con menos de 2 horas: " + menor2);

    System.out.println("\nEstudiantes por semestre:");
    // Ciclos anidados
    for (int s = 1; s <= 10; s++) {
        int contador = 0;
        for (int i = 0; i < n; i++) {
            if (semestre[i] == s) {
                contador++;
            }
        }
        System.out.println("Semestre " + s + ": " + contador);
    }

    sc.close();

```

}
}

5. Prueba de escritorio

Ejemplo con 4 estudiantes:

Estudiante	Edad	Semestre	Horas/día	¿< 2 h?
1	20	1	3.0	
2	22	2	1.5	
3	19	1	4.0	
4	25	3	0.5	

Cálculos: edad promedio = $(20+22+19+25)/4 = 21.50$ años; horas promedio = $(3+1.5+4+0.5)/4 = 2.25$ h; mayor cantidad = estudiante 3 con 4 h; menos de 2 h = 2 estudiantes.

Conteo por semestre: semestre 1 = 2; semestre 2 = 1; semestre 3 = 1; semestres 4–10 = 0.

EJERCICIO 10. Sistema integrador de parqueadero

Para completar los requisitos que no traen una tarifa concreta en el enunciado, se utiliza una propuesta de tarifas por rol: estudiante \$0.50/h, docente \$0.75/h y visitante \$1.00/h. El programa tiene capacidad máxima de 50 vehículos. Estas tarifas pueden cambiarse fácilmente en el código.

1. Análisis del problema

Inicio	Mostrar el menú del parqueadero y preparar arreglos, contadores y acumuladores.
Entrada	Opción del menú; tipo de vehículo; rol; número de horas; día de la semana; boleto perdido.
Proceso	Registrar y validar datos, calcular tarifa, guardar registros y generar estadísticas y recaudación.
Salida	Vehículos registrados, cantidades por tipo y rol, total de horas, promedio de permanencia, total recaudado, mayor y menor pago.
Fin	Elegir Salir y finalizar el programa.

2. Tarifas propuestas

Rol	Tarifa por hora	Ejemplo 2 horas
Estudiante	\$0.50	\$1.00
Docente	\$0.75	\$1.50
Visitante	\$1.00	\$2.00

3. Algoritmo

```
Inicio
    capacidad ← 50
    totalVehiculos ← 0
    Repetir
        Mostrar menú
        Leer opción

        Según opción:
        1. Registrar vehículo
            Si totalVehiculos < capacidad
                Leer y validar tipo, rol, horas, día y boleto
                Calcular tarifa según rol
                Si boleto perdido, agregar recargo de $5
                Guardar datos
                Actualizar contadores y acumuladores
                totalVehiculos ← totalVehiculos + 1
            Si no
                Mostrar "Parqueadero lleno"
            Fin Si

        2. Mostrar vehículos registrados
            Recorrer registros y mostrarlos

        3. Mostrar estadísticas
            Calcular promedio
            Contar por tipo y por rol
            Mostrar total de horas, promedio, mayor y menor pago

        4. Mostrar recaudación
            Mostrar total recaudado

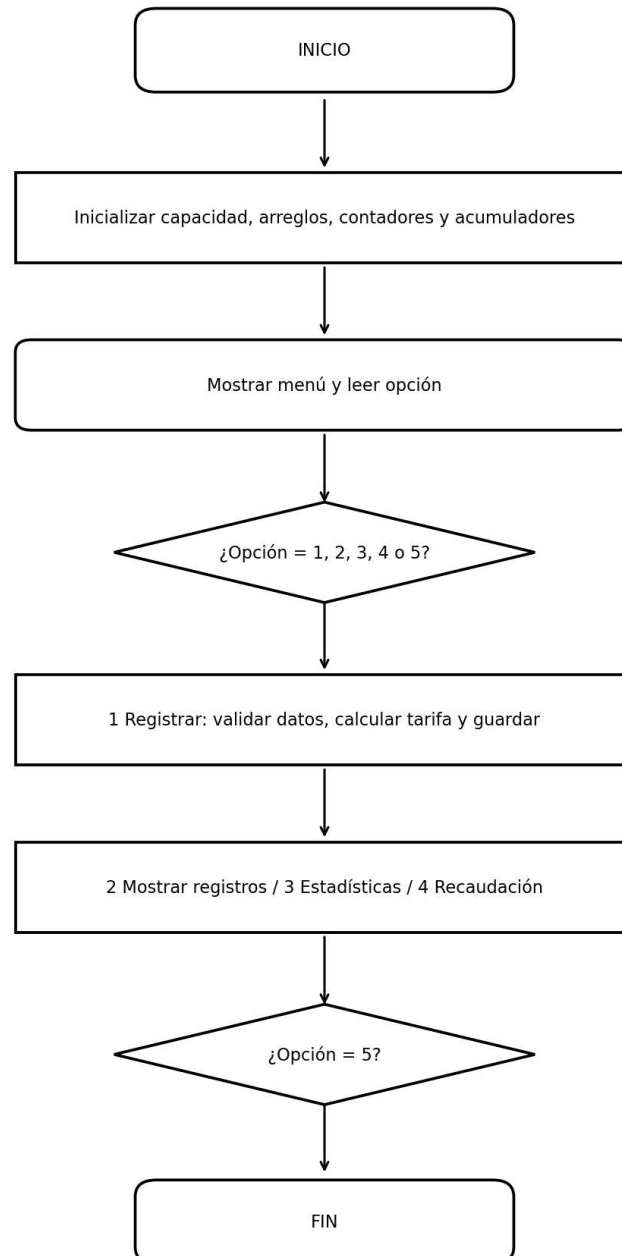
        5. Salir

    Otra opción → mostrar error
```

```
Hasta que opción = 5  
Fin
```

4. Diagrama de flujo

Ejercicio 10 - Sistema de parqueadero



La opción 1 contiene validaciones y el cálculo de la tarifa; las opciones 2–4 consultan la información almacenada.

5. Código Java

```
import java.util.Scanner;  
  
public class Ejercicio10 {  
    static final int MAX = 50;
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    String[] tipo = new String[MAX];
    String[] rol = new String[MAX];
    int[] horas = new int[MAX];
    int[] dia = new int[MAX];
    boolean[] perdido = new boolean[MAX];
    double[] pago = new double[MAX];

    int totalVehiculos = 0;
    int[] contTipo = new int[3]; // 0 carro, 1 moto, 2 bicicleta
    int[] contRol = new int[3]; // 0 estudiante, 1 docente, 2 visitante
    int totalHoras = 0;
    double totalRecaudado = 0;
    double mayorPago = 0;
    double menorPago = Double.MAX_VALUE;

    int opcion;
    do {
        System.out.println("\n=====");
        System.out.println("        PARQUEADERO UNIVERSITARIO");
        System.out.println("=====");
        System.out.println("1. Registrar vehículo");
        System.out.println("2. Mostrar vehículos registrados");
        System.out.println("3. Mostrar estadísticas");
        System.out.println("4. Mostrar recaudación");
        System.out.println("5. Salir");
        System.out.print("Opción: ");
        opcion = sc.nextInt();

        switch (opcion) {
            case 1:
                if (totalVehiculos >= MAX) {
                    System.out.println("Parqueadero lleno.");
                    break;
                }

                int t;
                do {
                    System.out.print("Tipo (1-Carro, 2-Moto, 3-Bicicleta): ");
                    t = sc.nextInt();
                } while (t < 1 || t > 3);

                int r;
                do {
                    System.out.print("Rol (1-Estudiante, 2-Docente, 3-Visitante): ");
                    r = sc.nextInt();
                } while (r < 1 || r > 3);

                int h;
                do {
                    System.out.print("Número de horas (1-24): ");
                    h = sc.nextInt();
                } while (h < 1 || h > 24);

                int d;
                do {
                    System.out.print("Día (1-Lun ... 7-Dom): ");
                    d = sc.nextInt();
                } while (d < 1 || d > 7);
            }
        }
    } while (opcion != 5);
}

```

```

int p;
do {
    System.out.print("¿Boleto perdido? (1-Sí, 2-No): ");
    p = sc.nextInt();
} while (p < 1 || p > 2);

tipo[totalVehiculos] = (t == 1) ? "Carro"
    : (t == 2) ? "Moto" : "Bicicleta";
rol[totalVehiculos] = (r == 1) ? "Estudiante"
    : (r == 2) ? "Docente" : "Visitante";
horas[totalVehiculos] = h;
dia[totalVehiculos] = d;
perdido[totalVehiculos] = (p == 1);

double tarifa;
switch (r) {
    case 1:
        tarifa = 0.50;
        break;
    case 2:
        tarifa = 0.75;
        break;
    default:
        tarifa = 1.00;
}

pago[totalVehiculos] = h * tarifa;
if (perdido[totalVehiculos]) {
    pago[totalVehiculos] += 5.00;
}

contTipo[t - 1]++;
contRol[r - 1]++;
totalHoras += h;
totalRecaudado += pago[totalVehiculos];

if (pago[totalVehiculos] > mayorPago)
    mayorPago = pago[totalVehiculos];
if (pago[totalVehiculos] < menorPago)
    menorPago = pago[totalVehiculos];

totalVehiculos++;
System.out.println("Vehículo registrado.");
break;

case 2:
    if (totalVehiculos == 0) {
        System.out.println("No hay vehículos registrados.");
    } else {
        for (int i = 0; i < totalVehiculos; i++) {
            System.out.printf(
                "%d) %s | %s | %d h | día %d | boleto perdido: %s | $%.2f%n",
                i + 1, tipo[i], rol[i], horas[i], dia[i],
                perdido[i] ? "Sí" : "No", pago[i]);
        }
    }
    break;

case 3:
    System.out.println("\n--- ESTADÍSTICAS ---");
    System.out.println("Vehículos registrados: " + totalVehiculos);
    System.out.println("Carros: " + contTipo[0]);
    System.out.println("Motos: " + contTipo[1]);

```

```

        System.out.println("Bicicletas: " + contTipo[2]);
        System.out.println("Estudiantes: " + contRol[0]);
        System.out.println("Docentes: " + contRol[1]);
        System.out.println("Visitantes: " + contRol[2]);
        System.out.println("Total de horas: " + totalHoras);

        if (totalVehiculos > 0) {
            System.out.printf("Promedio de permanencia: %.2f h\n",
                (double) totalHoras / totalVehiculos);
            System.out.printf("Mayor valor pagado: $%.2f\n", mayorPago);
            System.out.printf("Menor valor pagado: $%.2f\n", menorPago);
        }
        break;

    case 4:
        System.out.printf("Total recaudado: $%.2f\n", totalRecaudado);
        break;

    case 5:
        System.out.println("Programa finalizado.");
        break;

    default:
        System.out.println("Opción no válida.");
    }

} while (opcion != 5);

sc.close();
}
}

```

6. Prueba de escritorio

Ejemplo de tres registros usando las tarifas propuestas:

N°	Tipo	Rol	Horas	Día	Boleto	Tarifa	Pago
1	Carro	Estudiante	2	1	No	0.50/h	\$1.00
2	Moto	Docente	4	3	Sí	0.75/h	\$8.00
3	Bicicleta	Visitante	3	5	No	1.00/h	\$3.00

Resultados de la prueba: vehículos = 3; total de horas = 9; promedio de permanencia = 3.00 h; recaudación = \$12.00; mayor pago = \$8.00; menor pago = \$1.00. Por tipo: 1 carro, 1 moto y 1 bicicleta. Por rol: 1 estudiante, 1 docente y 1 visitante.

7. Casos límite considerados

- Cantidad máxima: 50 vehículos; al intentar registrar el vehículo 51 se muestra "Parqueadero lleno".
- Horas mínimas y máximas: 1 y 24.
- Días válidos: 1 a 7.
- Boleto perdido: agrega un recargo de \$5.00.
- Si no hay vehículos registrados, las estadísticas evitan dividir entre cero.
- Una opción del menú fuera de 1–5 se considera inválida.

Resumen de requisitos cumplidos

- Ejercicio 8: análisis, algoritmo, diagrama, código Java corregido y prueba de escritorio.
- Ejercicio 9: análisis, algoritmo, diagrama, validaciones, ciclos anidados, código Java y prueba de escritorio.
- Ejercicio 10: análisis, algoritmo, diagrama, código Java, switch, for, while, do-while, ciclos/recorridos, contadores, acumuladores, validaciones, casos límite y prueba de escritorio.